

Simulacro 2 – Técnicas de Diseño de Algoritmos (Segundo Parcial)

Duración sugerida: 3 horas

Total: 100 puntos

Ejercicio 1 – Multiple choice (10 puntos)

Para cada ítem elegir exactamente una opción. No es necesario justificar, salvo que el enunciado lo pida.

Cada respuesta correcta suma 2 puntos, respuesta incorrecta o con más de una opción marcada suma 0 puntos.

1. Sea un grafo no dirigido con n vértices y m aristas, representado con matriz de adyacencia.

¿Cuál es el costo asintótico de correr BFS desde un vértice origen s ?

- a) $O(n + m)$
- b) $O(n^2)$
- c) $O(m \log n)$
- d) $O(n)$

2. Sea un grafo no dirigido y ponderado con pesos positivos todos distintos. ¿Cuál de las siguientes afirmaciones es verdadera?

- a) El AGM es único y cualquier recorrido DFS que evite ciclos lo encuentra.
- b) El AGM es único y tanto Prim como Kruskal lo encuentran (con cualquier criterio de desempate).
- c) Pueden existir varios AGM distintos, incluso con todos los pesos diferentes.
- d) Si hay un único AGM, entonces los caminos mínimos entre todos los pares de vértices coinciden con los caminos en el AGM.

3. En un grafo dirigido ponderado con una arista de peso negativo y sin ciclos negativos, se corre Dijkstra desde un vértice origen s . ¿Qué puede ocurrir?

- a) El algoritmo siempre termina y las distancias calculadas son correctas.
- b) El algoritmo no termina.
- c) El algoritmo termina, pero puede devolver distancias incorrectas.
- d) El algoritmo detecta automáticamente la presencia de la arista negativa y la ignora.

4. Sea un DAG (grafo dirigido acíclico). Se realiza un DFS en el grafo. ¿Cuál de las siguientes afirmaciones es necesariamente verdadera?

- a) Todas las aristas son de árbol.
- b) No existen back edges.
- c) Todas las aristas son de avance.
- d) El orden de descubrimiento de los vértices da siempre un orden topológico.

5. Se tiene una red de flujo con capacidades enteras no negativas y se conoce el valor del flujo máximo F .

Si se multiplican todas las capacidades por una constante entera $k > 1$, ¿qué se puede afirmar sobre el valor del flujo máximo en la nueva red?

- a) Sigue siendo F .
- b) Pasa a ser F/k .
- c) Pasa a ser $k \cdot F$.
- d) No se puede concluir nada en general.

Tema que evalúa: complejidad de recorridos según representación, unicidad del AGM, condiciones de corrección de Dijkstra, propiedades de DFS en DAGs, efecto de escalar capacidades en flujo máximo.

Ejercicio 2 – AGM, caminos minimax y AGM vs caminos mínimos (20 puntos)

Sea un grafo no dirigido, conexo y ponderado $G = (V, E)$, con pesos positivos en las aristas. Para dos vértices $u, v \in V$, definimos el costo minimax del camino $u \Rightarrow v$ como:

$$\text{minimax}(u, v) = \min_{\{ \text{caminos } P \text{ de } u \text{ a } v \}} \max_{\{ e \in P \}} w(e),$$

es decir, queremos minimizar el peso máximo de una arista del camino.

1) (6 pts)

Diseñe un algoritmo que, dado G y dos vértices $s, t \in V$, calcule $\text{minimax}(s, t)$.

- Debe usar (explícitamente) un árbol generador mínimo de G .
- Describa el algoritmo a alto nivel (pseudocódigo o pasos claros).
- Analice su complejidad temporal en función de $|V|$ y $|E|$, asumiendo listas de adyacencia.

2) (8 pts)

Justifique por qué el algoritmo de (1) es correcto. Se permite una justificación informal pero clara, por ejemplo,

usando alguna de estas ideas:

- Propiedad de corte/ciclo de los AGM.
- Argumento de intercambio (“exchange argument”).

En particular, explique por qué el único camino entre s y t dentro de un AGM es efectivamente un camino minimax en el grafo original.

3) (6 pts)

Considere la siguiente afirmación:

“Dado un grafo no dirigido y ponderado G , el camino entre s y t dentro de cualquier árbol generador mínimo de G es siempre un camino de costo total mínimo (shortest path) entre s y t en el grafo original.”

- a) Indique si la afirmación es verdadera o falsa.
- b) En caso de ser falsa, describa un contraejemplo general (no hace falta dibujar, alcanza con nombrar vértices y pesos) donde se vea claramente que el camino en el AGM y el camino de suma mínima entre dos vértices difieren.

Tema que evalúa: construcción y uso del AGM, caminos minimax/maximin, relación (y diferencia) entre AGM y caminos mínimos, uso de propiedades de corte/ciclo y argumentos de corrección tipo greedy.

Ejercicio 3 – Caminos mínimos en DAGs y modelado de planificación (20 puntos)

Una empresa tiene un conjunto de tareas $T = \{1, \dots, n\}$. Cada tarea i tiene una duración $d(i) > 0$ y algunas tareas deben realizarse antes que otras.

Se modela esto como un DAG $G = (V, E)$ donde:

- Cada vértice $i \in V$ representa una tarea.
- Una arista dirigida $(i, j) \in E$ significa “la tarea i debe terminar antes de que pueda empezar la tarea j ”.

Se supone que la empresa tiene recursos ilimitados (puede ejecutar en paralelo cualquier cantidad de tareas siempre que se respeten las precedencias).

1) (6 pts)

Explique cómo modelar este problema como un problema de caminos en un DAG de forma que calcular el tiempo mínimo total

necesario para completar todas las tareas sea equivalente a calcular el valor de un camino (mínimo o máximo, según cómo lo modele).

- Especifique qué pesos coloca en las aristas o vértices.
- Aclare si termina resolviendo un problema de “camino mínimo” o de “camino máximo” en el DAG.

2) (8 pts)

Diseñe un algoritmo eficiente que, dada la entrada (el DAG de precedencias y la duración de cada tarea), calcule el tiempo

mínimo total para completar todas las tareas.

- Describa los pasos principales (es suficiente un pseudocódigo razonablemente claro).
- El algoritmo debe aprovechar que el grafo es un DAG (no use Dijkstra/Bellman-Ford como caja negra).
- Analice la complejidad temporal en función de $n = |V|$ y $m = |E|$.

3) (6 pts)

¿Qué ocurre si el grafo de precedencias no es un DAG (es decir, tiene al menos un ciclo)?

- Explique qué significan los ciclos en este contexto de planificación.
- Indique cómo su algoritmo podría detectar esta situación y qué debería responder.

Tema que evalúa: uso de DAGs para modelar problemas reales, diseño de DP sobre orden topológico, complejidad $O(n+m)$, interpretación de ciclos en grafos de precedencia.

Ejercicio 4 – Flujo máximo y modelado de asignación (30 puntos)

Una universidad quiere organizar talleres optativos para estudiantes. Se tiene:

- Un conjunto de estudiantes $S = \{s_1, \dots, s_n\}$.
- Un conjunto de talleres $W = \{w_1, \dots, w_m\}$.
- Cada taller w_j tiene un cupo máximo c_j (cantidad de estudiantes que puede aceptar).

- Cada estudiante si entrega una lista de talleres a los que está dispuesto a asistir (puede ser más de uno).
- Cada estudiante puede inscribirse como máximo en un taller.

El objetivo es inscribir la mayor cantidad posible de estudiantes en talleres compatibles con sus preferencias y respetando los cupos.

1) (10 pts)

Modele este problema como un problema de flujo máximo en una red.

Describa con precisión:

- Qué vértices tendrá la red (incluyendo origen y sumidero).
- Qué aristas se agregan (y entre qué vértices).
- Qué capacidades se asignan a cada tipo de arista.

No es necesario dibujar el grafo, pero la descripción debe ser lo suficientemente clara como para poder reconstruirlo.

2) (6 pts)

Explique por qué, en su modelo, cualquier flujo f entero desde el origen al sumidero induce una asignación válida “estudiante $\rightarrow$ taller” que respeta:

- Que cada estudiante esté inscripto a lo sumo en un taller.
- Que ningún taller supere su cupo.

Puede apoyarse en la propiedad de integridad de los algoritmos de flujo máximo, pero debe explicarla brevemente.

3) (8 pts)

Proponga un algoritmo para encontrar una asignación que maximice la cantidad de estudiantes inscriptos.

- Indique qué algoritmo de flujo máximo utilizaría (por ejemplo, Ford–Fulkerson, Edmonds–Karp, Dinic, etc.).
- Analice su complejidad temporal en función de n , m y la cantidad total de aristas que aparecen en la red construida.

4) (6 pts)

Extienda el modelo para el siguiente escenario: ahora, algunos estudiantes pueden asistir hasta a dos talleres diferentes

(el resto sigue pudiendo asistir a lo sumo a uno).

- Describa cómo debería modificar su red de flujo para capturar esta nueva restricción.
- No hace falta rediseñar todo desde cero, basta con explicar los cambios estructurales necesarios.

Tema que evalúa: modelado de problemas de asignación/matching con flujo, construcción de la red, interpretación del flujo máximo
como solución discreta, integridad de soluciones, complejidad de algoritmos de flujo y extensiones del modelo.

Ejercicio 5 – Preguntas cortas sobre caminos mínimos y elección de algoritmos (20 puntos)

Responder de forma breve y clara (3–5 líneas por inciso). Cada ítem vale 5 puntos.

1) (5 pts)

Explique en qué condiciones es correcto usar BFS para calcular caminos mínimos desde un origen s en un grafo.

- ¿Qué restricciones deben cumplir los pesos de las aristas?
- ¿Qué complejidad temporal tiene BFS en términos de n y m ?

2) (5 pts)

Sea un grafo dirigido ponderado que puede tener aristas de peso negativo, pero no tiene ciclos de peso negativo alcanzables desde s .

- Explique brevemente por qué Dijkstra puede devolver distancias incorrectas en este contexto.
- ¿Qué algoritmo usaría en su lugar para caminos mínimos desde un solo origen? Indique también su complejidad temporal.

3) (5 pts)

Describa brevemente el algoritmo de Bellman–Ford y explique:

- Qué problema resuelve (qué tipo de grafos admite).
- Cómo puede usarse para detectar la existencia de un ciclo de peso negativo alcanzable desde el origen.

4) (5 pts)

Compare el uso de Floyd–Warshall con el enfoque de correr Dijkstra desde varios orígenes para calcular caminos mínimos entre muchos pares de vértices.

- Mencione la complejidad temporal de Floyd–Warshall en función de n .
- Comente en qué situaciones conviene usar Floyd–Warshall y en cuáles conviene usar “varios Dijkstra”
(no hace falta una cota ultra precisa, basta con una explicación razonada).